PROJECT DOCUMENTATION

Echochamber
Local Network File Sharing

Zero-config, ephemeral file and text sharing for everyone on your WiFi.

Hamshad
Version 1.0 · 2026.05
Self-published

Contents

01 · EXECUTIVE SUMMARY

Executive Summary

Echochamber is a lightweight, zero-configuration file and text sharing tool designed for local networks. It ties shared content to the user's public IP address, making files instantly accessible to all devices on the same WiFi network, with automatic 1-hour expiration.

Key Takeaways

- Zero-config setup: automatic room discovery via network IP, no account required
- Ephemeral by default: all items auto-delete after 1 hour to preserve privacy
- Realtime sync: powered by Firebase for instant updates across all devices

QUESTIONS THIS DOCUMENT ANSWERS

- What is Echochamber and how does it work?
- How to deploy Echochamber on Vercel or self-host?
- What is the technical stack behind Echochamber?

02 · PRODUCT OVERVIEW

Product Overview

Echochamber solves the friction of sharing files between devices on the same local network without cloud uploads or account creation.

Core Value Proposition

Unlike cloud-based sharing tools, Echochamber operates entirely within your local network. Content is tied to your public IP address, so only devices on the same network can access shared items, eliminating external privacy risks.

Key Features

- Zero Config: Automatic room discovery via network IP; no manual setup required
- Ephemeral Storage: Items live for 1 hour (configurable via TTL_MS) then vanish permanently
- Private: No user accounts, no tracking, no cloud storage — all data stays on your local network
- Realtime Sync: Instant updates via Firebase Realtime Database and Storage
- Cross-Platform: Web-based, works on any device with a browser

Use Cases

Ideal for quick file transfers between colleagues in an office, sharing notes during meetings, or distributing assets to devices on the same WiFi without email or messaging apps.

03 · TECHNICAL ARCHITECTURE

Technical Architecture

Echochamber uses a Node.js backend with Firebase as the realtime data and storage layer, deployed as a serverless application on Vercel.

Tech Stack

Component	Technology
Backend	Node.js, Express
Realtime Database	Firebase Realtime Database
File Storage	Firebase Storage
Deployment	Vercel (serverless)
Frontend	Vanilla JS, HTML5, CSS3

Data Flow

1. User uploads file or pastes text via the web interface
2. Backend extracts user's public IP address and associates content with it
3. Content is stored in Firebase Storage (files) or Realtime Database (text)
4. TTL timer starts; content auto-deletes after configured duration (default 1 hour)
5. Other devices on the same IP network query Firebase and retrieve available content

Configuration

All configuration is done via environment variables. Required variables include `FIREBASE_SERVICE_ACCOUNT`, `FIREBASE_STORAGE_BUCKET`, and `FIREBASE_DATABASE_URL`. Optional variables like `TTL_MS` (default 3600000ms / 1 hour) and `CLEANUP_INTERVAL_MS` (default 60000ms / 60 seconds) tune expiration behavior.

04 · DEPLOYMENT GUIDE

Deployment Guide

Echochamber can be deployed to Vercel in one click, or self-hosted on any Node.js-compatible server.

Hosted Version

The official hosted version is available at https://echochamber-share.vercel.app/, maintained by Hamshad. No installation required — visit the URL and start sharing immediately.

Self-Hosting on Vercel

1. Fork or clone the GitHub repository
2. Copy `.env.example` to `.env` and fill in Firebase credentials
3. Import project to Vercel and set environment variables in project settings
4. Deploy — Vercel will auto-build and serve the application

Self-Hosting (Local)

```
git clone https://github.com/hamshad/echochamber.git
cd echochamber
npm install
cp .env.example .env
# Fill .env with your Firebase credentials
npm start
```

The application will run on port 3000 by default (configurable via `PORT` environment variable).

05 · APPENDIX

Appendix

A. References

- Echochamber Live Site: https://echochamber-share.vercel.app/
- GitHub Repository: https://github.com/hamshad/echochamber
- Firebase Documentation: https://firebase.google.com/docs
- Vercel Documentation: https://vercel.com/docs

B. Glossary

TTL - Time-to-live: the duration an item remains available before automatic deletion.

Realtime Database - Firebase's NoSQL cloud database that syncs data across all connected clients in real time.

C. Acknowledgements

Echochamber was built and is maintained by Hamshad. The project uses Firebase for backend services and Vercel for hosting.